Data Preprocessing

1. Introduction

Data preprocessing is a crucial step in machine learning that prepares raw data for model training and analysis. Real-world datasets often contain irrelevant attributes, categorical values, and inconsistencies that must be transformed into a suitable format before building a machine learning model.

In this project, several preprocessing techniques were applied to clean the dataset, convert categorical variables into numerical form, normalize numerical features, and prepare the data for training the Random Forest classification model used to predict loan default risk.

2. Dataset Loading

The dataset was first loaded into the Python environment using the **Pandas library**, which provides powerful data manipulation and analysis capabilities.

```
import pandas as pd
df = pd.read_csv("microfinance_final_project_dataset.csv")
df.head()
```

This step allows us to inspect the dataset and understand its structure.

3. Dataset Structure Analysis

To understand the dataset structure, the .info() function was used.

```
df.info()
```

This step provides important information such as:

- Number of rows and columns in the dataset
- Data types of each feature
- Memory usage
- Presence of missing values

Understanding the dataset structure helps determine the necessary preprocessing operations.

4. Missing Value Check

Missing values can negatively affect machine learning models. Therefore, the dataset was checked for missing values using:

df.isnull().sum()

The results showed that the dataset did not contain missing values, so no imputation or replacement was required.

5. Removing Irrelevant Features

The **applicant_id** column was removed from the dataset because it only represents a unique identifier and does not contribute to predicting loan default risk.

df = df.drop("applicant_id", axis=1)

Removing unnecessary attributes helps reduce noise and improves the efficiency of the model.

6. Encoding Categorical Variables

Machine learning algorithms require numerical input. However, the dataset contains categorical variables such as:

- Gender
- Region
- Education
- Employment
- Loan Purpose
- Repayment History
- Collateral

These categorical variables were converted into numerical form using **One-Hot Encoding**.

df = pd.get_dummies(df, drop_first=True)

One-hot encoding converts categorical variables into binary columns where:

- **1 indicates the presence of a category**
- **0 indicates the absence of a category**

This transformation enables the machine learning model to interpret categorical features effectively.

7. Feature and Target Variable Separation

After encoding the categorical variables, the dataset was divided into input features and the target variable.

```
X = df.drop("loan_status", axis=1)
y = df["loan_status"]
```

- **X (features)** contains all input variables used for prediction
- **y (target variable)** represents the loan status indicating whether the borrower defaults or not

8. Train-Test Split

To evaluate the performance of the machine learning model, the dataset was divided into training and testing sets.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
```

- **80% of the data** was used for training the model
- **20% of the data** was used for testing the model

This ensures that the model is evaluated on unseen data.

9. Feature Scaling

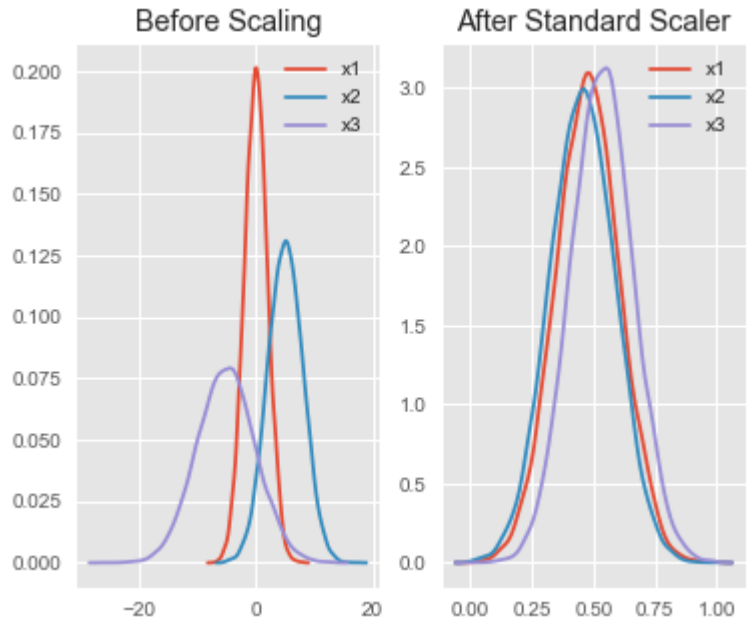
Feature scaling was applied to normalize numerical features so that all variables have a similar scale. This improves the performance and stability of machine learning algorithms.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

StandardScaler transforms the data so that:

- Mean ≈ **0**
- Standard deviation ≈ **1**

Feature Scaling Illustration:



10. Class Imbalance Check

The distribution of the target variable was analyzed to determine whether the dataset was balanced.

```
print(y.value_counts())
```

This step helps identify whether one class significantly dominates the dataset, which may influence model performance.

12. Saving the Preprocessed Dataset

After completing the preprocessing steps, the cleaned dataset was saved for further use.

```
df.to_csv("preprocessed_microfinance_dataset.csv", index=False)
```

This allows other team members to directly use the processed dataset for model training and evaluation.

13. Conclusion

The data preprocessing stage ensured that the dataset was clean, structured, and suitable for machine learning algorithms. Key preprocessing steps included removing irrelevant attributes, encoding categorical variables, splitting the dataset into training and testing sets, applying feature scaling, and checking class distribution.

These preprocessing techniques improved the quality of the dataset and prepared it for effective model training and evaluation.